

LLD Practice Platform — Specification & AI Usage Log

Complete platform guide, 10 canonical challenges, quickstart setup, REST API reference, and architectural AI trade-offs.

📅 Generated: September 2026 🏠 Project: LLD Practice Platform ⚡ Status: Verified & Production Ready

📖 LLD Practice Platform ### *Interactive Low-Level Design (LLD) Mastery & Interview Preparation Platform* Practice canonical object-oriented design problems, structure classes and responsibilities, visualize live Mermaid UML diagrams, and receive explainable deterministic & AI-assisted feedback across 9 design dimensions.

[[React]](https://img.shields.io/badge/React_19-20232A?style=for-the-badge&logo=react&logoColor=61DAFB)(https://react.dev/) [[TypeScript]](https://img.shields.io/badge/TypeScript-007ACC?style=for-the-badge&logo=typescript&logoColor=white)(https://www.typescriptlang.org/) [[Vite]](https://img.shields.io/badge/Vite_6-646CFF?style=for-the-badge&logo=vite&logoColor=white)(https://vitejs.dev/) [[TailwindCSS]](https://img.shields.io/badge/Tailwind_CSS_v4-38B2AC?style=for-the-badge&logo=tailwind-css&logoColor=white)(https://tailwindcss.com/) [[Python]](https://img.shields.io/badge/Python_3.13-3776AB?style=for-the-badge&logo=python&logoColor=white)(https://www.python.org/) [[Django]](https://img.shields.io/badge/Django_6.0-092E20?style=for-the-badge&logo=django&logoColor=white)(https://www.djangoproject.com/) [[DRF]](https://img.shields.io/badge/Django_REST_Framework-A30000?style=for-the-badge&logo=django&logoColor=white)(https://www.django-rest-framework.org/) [[License: MIT]](https://img.shields.io/badge/License-MIT-green.svg?style=for-the-badge)(LICENSE)

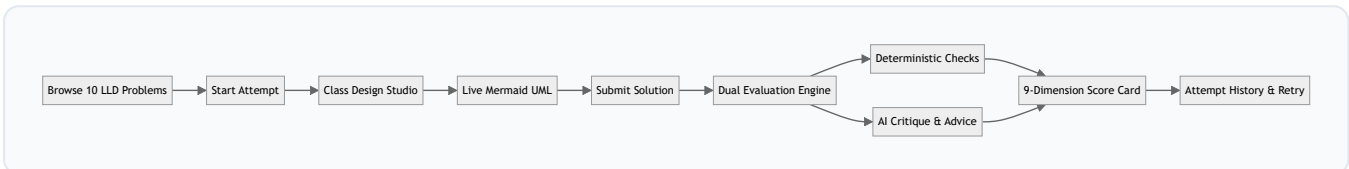
[🌟 Key Highlights](#-key-highlights) • [🌱 10 Canonical Problems](#-10-canonical-ll-d-challenges) • [🧑‍🎓 Practice Studio](#-interactive-practice-studio) • [💬 Evaluation Engine](#-dual-layer-evaluation-engine) • [🚀 Quick Start](#-quick-start-guide) • [📖 API Reference](#-rest-api-reference) ---

🌟 Key Highlights

- 🔄 **End-to-End Learner Journey:** Choose Problem → Analyze Requirements → Design Classes → Generate UML → Submit → Deterministic & AI Evaluation → Iterate & Improve .
- 📐 **Interactive Class Design Studio:** Add and edit classes, define Single Responsibility Principle (SRP) contracts, specify typed methods and attributes, and model relationships (Inheritance, Implementation, Composition, Aggregation).
- 📊 **Real-time Mermaid UML Visualizer:** Generates instant, clean UML class diagrams with one-click "Sync from Classes" capability.
- 🎯 **Dual-Layer Explainable Feedback Engine:**
- ⚙️ **Deterministic Rules Engine:** 8 automated static checks (Requirements Keyword Coverage, God Class Prevention, Polymorphism & Abstraction, Relationship Integrity, SOLID principles).

- **AI Reasoning Layer (DeepSeek / Gemini / OpenAI):** Context-aware qualitative critique, coupling analysis, and concrete architectural suggestions.
- **Zero-Failure Fallback:** Automatically operates in deterministic mode if AI API keys are not supplied.
-  **Attempt History & Progression Tracking:** Preserves past attempts, tracks score improvements, and supports iterative "Try Again" workflows.

Learner Workflow



10 Canonical LLD Challenges

Each problem comes pre-seeded with realistic interview requirements, edge cases, hints, and expected design patterns:

#	Problem	Difficulty	Est. Time	Key Patterns & Concepts Tested
01	Parking Lot System	 Medium	~45 min	Strategy (Pricing/Allocation), Factory, Polymorphic Spot Sizing
02	Elevator Control Bank	 Hard	~60 min	State Machine, SCAN/LOOK Dispatcher Strategy, Observer
03	Vending Machine System	 Easy	~30 min	State Pattern, Optimal Coin Change Calculation, Inventory Lock
04	Library Management	 Medium	~45 min	Book vs BookItem Abstraction, FIFO Reservation, Fine Strategy
05	Splitwise (Expense Sharing)	 Medium	~45 min	Equal/Exact/Percent Splits, Debt Minimization Graph Algorithm
06	Movie Ticket Booking	 Hard	~60 min	Temporary Seat Locking with TTL, Concurrency, Dynamic Pricing
07	Snake & Ladder Game	 Easy	~30 min	Board Entity Polymorphism, Pluggable Dice Strategy, Turn Manager
08	ATM System	 Medium	~45 min	State Pattern Lifecycle, Chain of Responsibility Note Dispenser
09	API Rate Limiter Library	 Hard	~60 min	Token Bucket & Sliding Window Strategies, Atomic Concurrency
10	Chess Game Engine	 Hard	~60 min	Polymorphic Piece Rules, Command Pattern Move History, Checkmate

Interactive Practice Studio

1. Class Design Studio - Define class name, package, type (Class, Abstract Class, Interface). - Add typed fields with visibility modifiers (`+public`, `-private`, `#protected`). - Define methods with parameter signatures and return types. - Set up relationships: - `--|>` Inheritance - `..|>` Interface Implementation - `\*-` Composition - `o--` Aggregation

2. Live Mermaid UML Studio - Instant visualization of classes and relationships as an interactive SVG diagram. - One-click **Sync from Classes** to auto-generate UML syntax. - Real-time syntax error validation. - Free-form architecture notes and Design Pattern tagging (Strategy, Factory, Observer, State, Command, etc.).

Dual-Layer Evaluation Engine

The platform evaluates solutions across **9 core dimensions**:

1. Requirements Coverage
2. God Class Prevention
3. Polymorphism & Abstraction
4. Coupling & Cohesion
5. SOLID Principles
6. Extensibility & Open-Closed
7. Appropriate Design Patterns
8. Data Modeling Integrity
9. Edge Case Resilience

Deterministic vs. AI-Assisted Evaluation



Tech Stack & Architecture

Layer	Technology	Details
Frontend	React 19 + TypeScript	High-performance modern UI with typed state management
Styling	Tailwind CSS v4	Clean dark/light modern developer aesthetic
Diagrams	Mermaid.js	Live client-side UML rendering
Backend	Python 3.13 + Django 6	Clean Architecture with Domain, Service, and API separation
API	Django REST Framework	RESTful endpoints with full serialization and validation
Evaluators	Extensible Evaluator Pattern	<code>RuleBasedEvaluator</code> , <code>AIEvaluator</code> , <code>CompositeEvaluator</code>
Testing	Pytest + Pytest-Django	Comprehensive unit, service, and API integration test suite

Quick Start Guide

Prerequisites

- **Python:** `3.10` or higher
- **Node.js:** `18.x` or higher
- **npm:** `9.x` or higher

1. Clone & Setup Repository

```
git clone https://github.com/ffhfgfh/LLD-Practice-Platform.git
cd LLD-Practice-Platform
```

2. Backend Setup (Django)

```
# Navigate to backend directory
cd backend

# Create virtual environment
python -m venv venv

# Activate virtual environment
# Windows:
.\venv\Scripts\activate
# macOS/Linux:
# source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Run migrations and seed the 10 canonical LLD problems
python manage.py makemigrations core
python manage.py migrate
python manage.py seed_data

# Start Django development server (default: http://127.0.0.1:8000)
python manage.py runserver
```

3. Frontend Setup (React + Vite)

In a new terminal window:

```
# Navigate to frontend directory
cd frontend

# Install npm packages
npm install

# Start Vite development server (default: http://localhost:5173)
npm run dev
```

Open <http://localhost:5173> in your browser.

4. Optional: Environment Configuration

Copy `.env.example` to `.env` in the root or `backend/` directory:

```
# Django Settings
SECRET_KEY=your-django-secret-key
DEBUG=True

# Evaluator Provider: 'composite' (default), 'rule_based', 'deepseek', 'gemini', 'openai'
EVALUATOR_TYPE=composite

# AI Provider Keys (Optional: Leave empty for 100% deterministic evaluation)
DEEPSEEK_API_KEY=your_deepseek_api_key_here
GEMINI_API_KEY=your_gemini_api_key_here
OPENAI_API_KEY=your_openai_api_key_here
```

Running Automated Tests

```
# Run backend test suite with Pytest
cd backend
pytest -v

# Run frontend build check
cd ../frontend
npm run build
```

REST API Reference

Method	Endpoint	Description
GET	/api/dashboard/	Learner progress summary, stats, and recommended problems
GET	/api/problems/	List all 10 problems (filter by <code>?difficulty=</code> or <code>?search=</code>)
GET	/api/problems/:id/	Detailed requirements, constraints, hints, and diagrams
POST	/api/attempts/	Initialize a new practice attempt (<code>{"problem_id": "<id>"}</code>)
GET	/api/attempts/:id/	Fetch current attempt state, draft solution, and classes
PUT	/api/attempts/:id/	Auto-save draft classes, diagram, and architectural notes
POST	/api/attempts/:id/submit/	Submit solution and execute evaluation pipeline
GET	/api/evaluations/:id/	Get evaluation report, 9–category scores, and recommendations
GET	/api/history/	View past attempt history grouped by problem

📁 Project Structure

```
LLD-Practice-Platform/
├── backend/
│   ├── lld_platform/          # Django project settings & URL configuration
│   ├── core/
│   │   ├── domain/          # Pure domain layer (Dataclasses, Enums, Interfaces)
│   │   │   ├── enums.py      # Difficulty, AttemptStatus, EvaluationCategory
│   │   │   ├── models.py     # DomainProblem, DomainSolution, DomainClass
│   │   │   └── interfaces.py  # SolutionEvaluator interface
│   │   ├── models.py         # Django ORM models
│   │   ├── serializers.py    # DRF serializers
│   │   ├── views.py          # REST API ViewSets & endpoints
│   │   ├── urls.py           # API routing
│   │   └── services/         # Business logic & orchestrators
│   │       ├── attempt_service.py  # Attempt state & immutability lifecycle
│   │       ├── evaluation_service.py # Evaluation pipeline runner
│   │       └── evaluators/        # Pluggable evaluator strategies
│   │           ├── rule_based.py   # Deterministic 8-pass static analyzer
│   │           ├── ai_evaluator.py # DeepSeek / Gemini / OpenAI adapter
│   │           └── composite_evaluator.py
│   │       └── factory.py
│   ├── management/commands/
│   │   └── seed_data.py        # Seeder for 10 complete LLD challenges
│   └── tests/                  # Automated test suite (API, services, evaluators)
├── manage.py
├── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── common/        # Badge, Button, Card, Spinner, ErrorAlert, StatCard
│   │   │   ├── layout/        # Navbar, Footer, Layout wrapper
│   │   │   ├── problems/      # ProblemCard, RequirementsList, Details
│   │   │   ├── practice/      # ClassCard, ClassEditorModal, MermaidViewer, ArchitectureEditor
│   │   │   ├── evaluation/    # ScoreCard, CategoryScores, Strengths, FeedbackList
│   │   │   └── history/       # AttemptHistoryCard, AttemptComparisonModal
│   │   ├── pages/             # Dashboard, Problems, Practice, Feedback, History
│   │   ├── services/api.ts    # Axios REST API client
│   │   ├── types/index.ts     # TypeScript domain definitions
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── package.json
│   └── vite.config.ts
├── .env.example
├── .gitignore
├── DESIGN.md
├── RESEARCH.md
├── AI_USAGE.md
└── README.md
```

🍌 Contributing & License

Contributions, problem additions, and feedback are always welcome!

Feel free to open an [Issue](#) or submit a [Pull Request](#).

Distributed under the **MIT License**. See [LICENSE](#) for more information.

Built with ❤️ for software engineers mastering Object-Oriented & Low-Level Design.

AI_USAGE.md — Engineering Decisions & AI Collaboration Log

This document records 4 meaningful architectural and technical decisions made during the design and implementation of the **LLD Practice Platform**, contrasting AI-assisted recommendations against final engineering trade-offs.

Decision 1: AI Evaluation Dependency & Graceful Fallback Architecture

- **Initial AI Suggestion:** Make the entire evaluation flow dependent on an external LLM API (such as OpenAI GPT-4o or Google Gemini), calling the LLM directly within the Django REST API view and returning the raw generated markdown string.
 - **Accepted / Rejected:** REJECTED.
 - **Engineering Rationale:**
 1. Relying exclusively on an external LLM introduces a hard failure mode whenever a learner has no API key configured, experiences network timeouts, or encounters API rate limits.
 2. Directly calling third-party APIs inside API views tightly couples the transport layer to the external vendor and makes automated unit testing difficult.
 - **Final Implementation:**
 - Implemented the `SolutionEvaluator` abstraction with a deterministic `RuleBasedEvaluator` and a provider-agnostic `AIEvaluator`.
 - Created `CompositeEvaluator` which always runs deterministic static checks first, attempts AI qualitative reasoning, and gracefully falls back to deterministic results with `is_fallback=True` if the AI service fails.
-

Decision 2: Solution Input Modeling (Structured Entities vs Free-form Code / Canvas)

- **Initial AI Suggestion:** Provide a full-blown canvas with draggable UML class nodes (like React Flow) or a raw Java/C++ code editor with language server protocol (LSP) compilation.
- **Accepted / Rejected:** REJECTED.
- **Engineering Rationale:**
 1. A raw code editor forces the platform to parse entire programming language ASTs, introducing compiler-specific syntax errors that distract from object-oriented domain thinking.
 2. An interactive drag-and-drop canvas adds heavy front-end state synchronization complexity without guaranteeing structured semantic data for automated evaluation.
- **Final Implementation:**
 - Designed a hybrid input model: structured class cards collecting class names, single responsibilities, typed methods, attributes, and relationships, combined with free-form architecture markdown, assumptions, and design pattern multi-selects.

- Added a live Mermaid class diagram renderer with an automated "**Sync from Classes**" button, giving learners visual UML representation without fragile canvas state.
-

Decision 3: Deterministic vs AI Category Score Blending

- **Initial AI Suggestion:** Let the LLM output the final category scores (0–10) and overall score (0–100) directly, overriding all deterministic calculations.
 - **Accepted / Rejected:** REJECTED.
 - **Engineering Rationale:**
 1. LLMs exhibit non-deterministic variance and can be easily prompt-injected or hallucinate that a requirement is covered when no such class exists in the submission.
 2. Ground-truth structural metrics (e.g. presence of God classes with > 7 methods, dangling class relationship references, missing responsibilities, requirement keyword matches) are mathematically verifiable and should anchor the evaluation.
 - **Final Implementation:**
 - Implemented a weighted composite formula: **40% Deterministic Verification + 60% AI Qualitative Reasoning**.
 - In the UI, feedback items are explicitly tagged as either **Deterministic Check** (teal badge) or **AI Reasoning** (indigo badge) so learners clearly understand the provenance of each critique.
-

Decision 4: Attempt Immutability & Iterative Learning Revision Model

- **Initial AI Suggestion:** Allow learners to continuously edit the same attempt in-place, overwriting the previous evaluation results with each save.
- **Accepted / Rejected:** REJECTED.
- **Engineering Rationale:**
 1. Overwriting attempts destroys historical learning progression and prevents learners from comparing their initial naive design with their subsequent refactored architecture.
 2. In engineering interviews, iterative progression (identifying bottlenecks in Attempt #1 and fixing them in Attempt #2) is the primary learning signal.
- **Final Implementation:**
 - Enforced attempt immutability at the domain service layer: once an attempt is in **SUBMITTED**, **EVALUATING**, or **COMPLETED** state, modifications to its classes or solution text are rejected with HTTP 400.
 - Implemented the "**Try Again**" workflow which creates a new **PracticeAttempt** with **attempt_number = N + 1** for the same problem, paired with an **Attempt History & Progression Modal** to compare scores across iterations.